🍴 ChefCoach

A Duolingo-Style Cooking Coach Agent (Skill Progression + Personalized Coaching)

Project Goal

Build an AI agent that teaches cooking as a curriculum (skills → practice → feedback → level-up), instead of only recommending a recipe. Maintain a structured learner profile (skill vector + preferences + constraints) and use it to plan each next lesson.

What the agent outputs each session:

- A target skill + micro-lesson
- A practice recipe/task matched to current level
- Step-by-step coaching + common mistakes + safety notes

Learning loop:

Collect user feedback (difficulty + taste + success/failure) and adapt future lessons ("RLHF-inspired", not full RLHF training).

Deliverable:

A small web app showing Session 1 → feedback → Session 2 improved plan, plus a visible skill progression dashboard.

Additional Data

Base LLM (choose 1 path/both)

Online LLM (recommended)

OpenAI/Claude-style API model
for coaching + judging

Local LLM (optional fallback)

Llama/Qwen/DeepSeek instruct via Ollama/vLLM
Lower cost, lower quality

Tier-1 Data (small & implementable)

Data	Size	Purpose
Food.com recipes (subset)	20K–50K	Recipe corpus for RAG (title + ingredients + steps + tags + ratings)
Skill Graph	8 skills (JSON)	Core skills + prerequisite rules (project-built, manual design)
User interaction data	Our own	Session logs + feedback labels (thumbs/difficulty/taste) in SQLite/JSON

Tier-2 (optional)

Cooking Q&A snippets (technique explanations) to support "why" questions.

System Architecture

1. **UI Input**
   Goal, time, ingredients, diet
2. **Profile Manager**
   Structured memory (JSON)
3. **Skill Assessor**
   Quiz → skill_vector + level
4. **Curriculum Planner**
   Skill Graph → next lesson
5. **RAG Retriever**
   Chroma/FAISS → recipe packets
6. **Coach Generator**
   LLM → steps + fixes + quiz
7. **Feedback → Update**
   RLHF-inspired loop

↻

Input → Profile → Assessment

User goal ("learn stir-fry"), time, ingredients, equipment, dietary constraints
UserProfile.json: skill_vector, taste_prefs, constraints, equipment, history[]
Lightweight assessment: short quiz + self-report + last-session outcome
Output: estimated skill_vector + current level (Lv1/Lv2/Lv3)

Curriculum → RAG → Coach Generation

Choose next target skill: prerequisites + weakest-skill strategy
Select lesson type: micro-lesson + practice recipe task
RAG: chunk by title/ingredients/steps separately → retrieve top-k
→ assemble "recipe packet" by recipe_id (full context stitched back)
LLM prompt = learner profile + curriculum state + retrieved context (in-context learning)
Output: objective, mise-en-place, step-by-step, timers, "if it goes wrong" fixes, reflection Qs

Feedback → Profile Update (RLHF-inspired)

Feedback: rating, difficulty, notes ("too spicy/too bland"), completion status
Update rules:
skill[s] = clip(skill[s] + lr * success_signal, 0, 1)
pref[p] = clip(pref[p] + lr * delta, 0, 1)
Emphasis: inspired by RLHF concept, not training reward model/PPO

Storage: UserProfile.json | InteractionMemory (SQLite) | Vector DB (Chroma/FAISS)

Evaluation Methods

A Pairwise LLM-as-Judge

Compare 3 systems:
1) No memory (single-turn coach)
2) Memory but no curriculum planner
3) ChefCoach (memory+curriculum+RLHF)

Rubric (1–5): personalization, difficulty, clarity, safety, teaching value
Report: win-rate + avg rubric scores

B Curriculum Progression Test

Simulate 3-session learning trajectories for multiple personas
Measure: does system unlock appropriate next lessons when skill > threshold?

C Grounding Spot Check

Random sample (~30 sessions): verify generated steps are consistent with retrieved recipe packets
Track: grounding issues, unsafe suggestions, missing steps

D Small User Study (optional)

5–10 users do 2–3 sessions
Collect: perceived learning improvement, satisfaction, "would cook again" rate

UI Plan (Streamlit)

Onboarding / Profile

Skill self-report
Dietary constraints
Equipment
Preferred cuisines

Lesson & Practice Chat

Current target skill
Practice recipe
Step-by-step guidance
"I'm stuck" button

Feedback

Thumbs up/down
Difficulty slider
Short text note

Progress Dashboard

Skill vector (radar/bar)
Unlocked lessons
Session timeline

Demo Script (2 min)

Session 1 (Lv1): Onboarding → first lesson → user feedback ("too bland", "steps too hard")
Session 2: Improved lesson + adjusted difficulty (shows personalization working)
Dashboard: Skill increase visible + next lesson unlocked

Tech Stack

LLM: Cloud API or local (Llama/Qwen/DeepSeek) | Vector DB: Chroma/FAISS | Storage: SQLite + JSON | UI: Streamlit